POLITECNICO
MILANO 1863

Prova Finale di Ingegneria Del Software

Client-Server Communication Protocol Documentation

Professor: Gianpaolo Saverio Cugola

Academic Year 2024 - 2025

Authors:

Samuele Segrini, Manuela Terribile, Alessandra Varricchio, Diego Zanca

Index

1. Introductions

1.1 Purpose

This document details the client-server communication protocol for the "Galaxy Trucker" game. It outlines the network architecture, message structures, and the sequence o interactions between the client and server components. The primary goal is to provide a comprehensive technical reference for understanding and maintaining the communication layer of the application

1.2 System Overview

The Galaxy Trucker game is a distributed application employing a client-server architecture. Clients can connect to a central server to participate in game sessions, which involve various phases such as lobby management, ship building, and flight. Communication between clients and the server is facilitated by a set of well-defined messages exchanged over the network. The system supports multiple concurrent game sessions and players. The client application offers both a Graphical User Interface (GUI) using JavaFX and a Terminal User Interface (TUI), providing flexibility for users. The server manages game logic, state, and synchronizes information across all connected clients within a session.

2. Network Architecture

The network architecture is designed to be modular and support multiple communication technologies. It relies on a clear separation of concerns between network handling, message dispatch, and application logic.

2.1 Communication Technologies

The system supports two primary network communication technologies:

- **TCP/IP Sockets**: Utilizes Java's java.net.Socket and java.net.ServerSocket for reliable, stream-based communication. Message objects are serialized and deserialized using java.io.ObjectInputStream and java.io.ObjectOutputStream.
- **Java RMI (Remote Method Invocation)**: Allows objects on one Java Virtual Machine (JVM) to invoke methods on objects in another JVM. This is used for a more object-oriented approach to remote communication.

The choice of technology can be configured at runtime, typically at client startup

2.2 Core Network Components

2.2.1 Client-Side Network Components

- ClientNetworkInterface: An interface defining the contract for client-side network adapters. It abstracts the specific technology used for communication (connect(), disconnect(), sendMessage(), etc.).
- SocketClientAdapter: Implements ClientNetworkInterface using TCP/IP Sockets. It manages a Socket connection and handles object serialization for sending and receiving Message objects. It uses a dedicated thread (socket-client-listener) to continuously listen for incoming messages.
- RMIClientAdapter: Implements ClientNetworkInterface using Java RMI. It acts as an IClientRemoteListener (a remote object itself) to receive callbacks from the server and interacts with the IServerRemote stub obtained from the RMI registry. Connection management includes registering with the server and handling remote exceptions. It employs an executor for sending commands and a scheduler for pinging the server to maintain connection liveness.

- ClientNetworkManager: Orchestrates client-side network operations. It holds an instance of a ClientNetworkInterface (either SocketClientAdapter or RMIClientAdapter). It sets up callbacks for message reception and disconnection, forwarding received messages to the client's EventBus. The sendMessage() method ensures messages are dispatched through the active network adapter.

2.2.2 Server-Side Network Components

- ServerNetworkInterface: An interface defining the contract for server-side network adapters. It abstracts how the server listens for connections and communicates with multiple clients (startServer(), stopServer(), sendMessageToClient(), etc.).

- SocketServerAdapter: Implements ServerNetworkInterface using TCP/IP Sockets. It uses a ServerSocket to listen for incoming client connections on a specified port. Upon accepting a connection, it spawns a new ClientHandler thread to manage communication with that specific client, using ObjectInputStream and ObjectOutputStream. Each client is assigned a unique clientId. : Implements ServerNetworkInterface and IServerRemote using Java RMI. It registers itself with an RMI registry, allowing clients to look it up. It manages a map of connected RMI clients (IClientRemoteListener stubs) and their session tokens.

- ServerNetworkManager: Manages multiple ServerNetworkInterface adapters (e.g., one for Sockets, one for RMI). It initializes and starts these adapters. It maintains a mapping from a globally unique clientId to the specific adapter that handles that client. This allows the server logic to send messages to a client without needing to know which network technology the client is using. It also provides global callbacks for client connection and disconnection events.

2.3 Message Handling and Dispatch

- Serialization: All messages implement java.io.Serializable, allowing them to be converted into a byte stream for network transmission and reconstructed at the receiving end. Both Socket and RMI mechanisms leverage Java's built-in serialization.

- EventBus: Both client and server utilize an EventBus for decoupled, asynchronous message handling.

  - On the client, when the ClientNetworkManager receives a message via its adapter, it posts the message to the clientEventBus.
  - On the server, when a ServerNetworkInterface (e.g., SocketServerAdapter's ClientHandler or RMIServerAdapter) receives a message from a client, it wraps it in an IncomingClientMessage (which includes the clientId) and posts it to the serverEventBus.

- @MessageHandler: Components interested in specific message types (e.g., ClientViewModel on the client, LoginController or GameSession on the server)

register methods annotated with @MessageHandler with their respective EventBus. The EventBus then invokes these methods when a matching message type is posted.

-

2.4 Connection Management

- Client Connection:
  - Socket: The client initiates a TCP connection to the server's IP and port. The server accepts this, and a dedicated communication channel is established.
  - RMI: The client looks up the IServerRemote stub in the RMI registry. It then calls registerClient() on this stub, passing its own IClientRemoteListener implementation. The server returns a unique session token.
- Client Identification: Each connected client is assigned a unique networkClientId by the ServerNetworkManager (or its adapters). This ID is used internally by the server to route messages and manage client state. For RMI, this is often the session token.
- Disconnection Handling:
  - If a client actively disconnects or a network error occurs (e.g., socket closure, RemoteException), the respective network adapter detects this.
  - The adapter then invokes its onDisconnectedCallback, which in turn triggers the ClientNetworkManager's (client-side) or ServerNetworkManager's (server-side) disconnection logic.
  - On the server, the LoginController is notified to handle the client's departure from any game sessions.
  - On the client, the ClientViewModel is updated to reflect the disconnected state.
- Pinging (RMI): To ensure RMI connections remain active and detect unresponsive clients/servers:
  - The RMIClientAdapter periodically calls pingServer() on the IServerRemote stub.
  - The server (via RMIServerAdapter) can, if needed, call pingClient() on the IClientRemoteListener stubs it holds. (Your current 'pingClient' implementation in 'RMIClientAdapter' doesn't do much other than check connectivity, but the mechanism exists). Failures in pinging can lead to connection termination.

3. Message Structure

All messages exchanged between the client and server adhere to a common structure defined by base classes and interfaces.

3.1 Message Interface

Located in it.polimi.ingsw.common.message, this is a marker interface.

```java
package it.polimi.ingsw.common.message;

import java.io.Serializable;

/**
 * Marker interface for all messages/events in the system.
 */
public interface Message extends Serializable { }
```

- Purpose: Serves as a common supertype for all messages, ensuring they are Serializable for network transmission.
- Serialization: Implementing Serializable allows Java's object serialization mechanism to convert message objects into a byte stream and vice-versa.

3.2 AbstractMessage Class

Located in it.polimi.ingsw.common.message, this class provides common functionality for most messages.

- Inheritance: Most concrete message classes extend AbstractMessage
- serialVersionUID: A version identifier for Serializable classes, crucial for ensuring compatibility during deserialization if class versions change.
- timestamp: A long value automatically set to System.currentTimeMillis() upon message creation. This can be used for logging, ordering, or debugging purposes.

3.3 Payload and Data Transfer Objects (DTOs)

Many messages carry additional data, often referred to as the payload. This payload is typically composed of:

- Primitive types (e.g., boolean, String, int).
- Data Transfer Objects (DTOs) from the it.polimi.ingsw.common.dto package. These DTOs are records or simple classes designed to encapsulate data for transfer, ensuring they are also Serializable. Examples include PlayerInfoDTO, ComponentDTO, GameLobbyInfoDTO, etc.

The specific fields constituting the payload are defined within each concrete message class.

4. System Messages

System messages handle fundamental interactions like login, error reporting, and basic notifications.

4.1 Client to Server

- **LoginRequest** : Sent by the client to request login to the server with a chosen nickname.

```
{
  "type": "LoginRequest",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": "nickname"
}
```

- **ReconnectRequest** : Request to reconnect to an existing session.

```
{
  "type": "ReconnectRequest",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": {
     "value":  "playerId",
     "value":  "sessionToken"
     }
}
```

4.2 Server to Client

- **LoginResponse** : Sent by the server in response to a ClientLoginRequest, indicating success or failure of the login attempt.

```
{
  "type": "LoginResponse",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload":{
     "value":  "correlationId",
```

```
        "value":  "playerId" ,
        "value":  "nickname"
        }
}
```

- **ErrorResponse** : A generic message used by the server to inform the client of an error condition.

```
{
   "type": "ErrorResponse",
   "RequestId": "RequestId",
   "CorrelationId": "CorrelationId",
   "payload": {
        "value":  "correlationId",
        "value":  "errorMessage",
        "value":  "errorCode"
        }
}
```

- **ReconnectResponse** : Response to a reconnection request.

```
{
   "type": "ReconnectResponse",
   "RequestId": "RequestId",
   "CorrelationId": "CorrelationId",
   "payload": {
        "value":  "correlationId",
        "value":  "playerId" ,
        "value":  "nickname" ,
        "value":  "gameId" ,
        "value":  "gameState" ,
        }
}
```

5. Set up Phase Messages

These messages are used during the game setup phase, including creating, listing, joining, and managing game lobbies.

5.1 Client to Server

- **CreateGameRequest** : sent by a player who wants to create a new game.

```
{
  "type": "CreateGameRequest",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": {
     "value":  "maxPlayers",
     "value":  "gameLevel",
     "value":  "gameName"
     }
}
```

- **ListGamesRequest** : sent by a player who wants a list of the already existing games.

```
{
  "type": "ListGamesRequest",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": "context"
}
```

- **JoinGameRequest** : sent by a player who wants to access an already existing game.

```
{
  "type": "JoinGameRequest",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": "gameId"
}
```

- **LeaveGameRequest** : sent by a player to requests to leave their current game session.

```
{
  "type": "LeaveGameRequest",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": "gameId"
}
```

- **SetPlayerReadyRequest** : Client updates their ready status within a game lobby.

```
{
  "type": "SetPlayerReadyRequest",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": "ready"
}
```

- **StartGameRequest** : sent by the host who wants to start the game.

```
{
  "type": "StartGameRequest",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": "gameId"
}
```

5.2 Server to Client

- **CreateGameResponse** : sent by the Server to indicate success or failure of game creation.

```
{
  "type": "CreateGameResponse",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload":{
     "value":  "correlationId",
     "value":  "gameId",
     "value":  "gameName"
     }
}
```

- **ListGamesResponse** : sent by the Server to confirm that a list of the already existing games is showing.

```
{
  "type": "ListGamesResponse",
  "RequestId": "RequestId",
```

```
  "CorrelationId": "CorrelationId",
  "payload":{
     "value":  "correlationId",
     "value":  "games"
     }
}
```

- **JoinGameResponse** : sent by the server to signal whether the access to an already existing game succeeded or failed.

```
{
  "type": "JoinGameResponse",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": {
     "value":  "correlationId",
     "value":  "gameInfo"
     }
}
```

- **StartGameResponse** : Response to start game request.

```
{
  "type": "StartGameResponse",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload": {
     "value":  "correlationId",
     "value":  "gameId"
     "value":  "gameModel"
     }
}
```

- **LeaveGameResponse** : sent by the server to signal whether the player left the game with success or not.

```
{
  "type": "LeaveGameResponse",
  "RequestId": "RequestId",
  "CorrelationId": "CorrelationId",
  "payload":"correlationId"
}
```

6. Ship Building Phase Messages

These messages facilitate the ship building phase of the game.

6.1 Client to Server

- **RequestFaceUpTileRequest** : sent by the client to request to take a face-up component tile.

```
{
     "type": "RequestFaceUpTileRequest",
     "RequestId": "RequestId",
     "CorrelationId": "CorrelationId",
     "payload": "tileId"
}
```

- **TakeTileRequest**: sent to pick a component tile from the central face-down pile.

```
{
     "type": "TakeTileRequest"
     "RequestId": "RequestId",
     "CorrelationId": "CorrelationId",
     "payload": false
}
```

- **ReserveTileRequest**: sent to server a component tile.

```
{
     "type": "ReserveTileRequest"
     "RequestId": "RequestId",
     "CorrelationId": "CorrelationId",
     "payload": "tileId"
}
```

- **PlaceTileRequest**: sent to place the component on the ship board.

```
{
     "type": "PlaceTileRequest",
     "RequestId": "RequestId",
     "CorrelationId": "CorrelationId",
```

```
    "payload": {
        "value":  "tileId",
        "value":  "row",
        "value":  "col",
        "value":  "rotation"
        }
```

- **ReturnTileRequest**: Client returns a component (that they were "holding" or took from reserve but decided not to place) to the common face-up pile.

```
{
        "type": "ReturnTileRequest",
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": "tileId"
}
```

- **FlipBuildingTimerRequest**: sent to flip the hourglass on the game board.

```
{
        "type": "FlipBuildingTimerRequest"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": false
}
```

- **ValidateShipRequest**: sent by the Client to request the validation of their ship.

```
{
        "type": "ValidateShipRequest"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": false
}
```

6.2 Server to Client

- **RequestFaceUpTileResponse**: sent by the Server to offer a drawn component to the player who requested it. The player can then decide to place, reserve, or return it.

```
{
        "type": "RequestFaceUpTileResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value":  "correlationId",
            "value":  "tile"
            }
}
```

- **TakeTileResponse**: sent by the Server to offer a drawn component to the player who requested it. The player can then decide to place, reserve, or return it.

```
{
        "type": "TakeTileResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
           "value":  "correlationId",
           "value":  "componentData"
           }
}
```

- **ReserveTileResponse**: Response to a reserve tile request.

```
{
        "type": "ReserveTileResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": "correlationId"
}
```

- **PlaceTileResponse**: Confirms that a component was successfully placed on a player's ship board.

```
{
        "type": "PlaceTileResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": "correlationId"
}
```

- **ValidateShipResponse**: Informs a player about the validation result of their ship component placement or overall ship structure.

```
{
        "type": "ValidateShipResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
           "value":  "correlationId",
           "value":  "errors"
           }
}
```

- **FlipBuildingTimerResponse**: Notifies players about changes to the building timer's state.

```
{
        "type": "FlipBuildingTimerResponse"
        "RequestId": "RequestId",
```

```
        "CorrelationId": "CorrelationId",
        "payload": {
            "value":  "correlationId",
            "value":  "newTimeRemaining"
            }
}
```

- **SetPlayerReadyResponse**: Notifies all players that a player has finished building and secured a flight order position.

```
{
        "type": "SetPlayerReadyResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value":  "correlationId",
            "value":  "ready"
            }
}
```

7. Flight Phase Messages

These messages facilitate the flight phase of the game.

7.1 Client to Server

- **DeclareStrengthRequest**: Request from a player to declare strength for an action, such as powering up engines or cannons, by committing batteries.

```
{
        "type": "DeclareStrengthRequest"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
           "value":  "decisionType",
           "value":  "batteriesToUse"
           }
}
```

- **LandPlanetRequest**: Request from a player to land on a planet (after PlanetsCard).

```
{
        "type": "LandPlanetRequest"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload":  "planetIndex"
```

- **DockRequest**: Request from a player to dock after either AbandonedShipChard or AbandonedStationCard.

```
{
        "type": "DockRequest"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload":  "isDocking"
```

7.2 Server to Client

- **CombatZoneResponse**: Response to a successful DeclareStrengthRequest (following a CombatZoneCard).

```
{
        "type": "CombatZoneResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value": "usedBatteries",
            "value": "lostFlightDays"
            "value": "lostCrew"
            }
}
```

- **LandPlanetResponse**: Response to a successful land on a planet request.

```
{
        "type": "LandPlanetResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value": "lostFlightDays",
            "value": "collectedGoods"
            }
}
```

- **MeteorSwarmResponse**: Response to a successful DeclareStrengthRequest (following a MeteorShowerCard)

```
{
        "type": "MeteorSwarmResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value": "usedBatteries",
            "value": "destroyedComponents"
            }
}
```

- **OpenSpaceResponse**: Response to a successful DeclareStrengthRequest (following an OpenSpaceCard).

```
{
        "type": "OpenSpaceResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value": "newFlightPosition",
            "value": "usedBatteries"
            }
```

```
}
```

- **PiratesLossResponse**: Response to a successful DeclareStrengthRequest (following the loss to a PiratesCard).

```
{
        "type": "PiratesLossResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value":  "usedBatteries",
            "value":  "destroyedComponents"
            }
}
```

- **PiratesResponse**:Response to a successful DeclareStrengthRequest (following a PiratesCard).

```
{
        "type": "PiratesResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value":  "usedBatteries",
            "value":  "hasWon"
            "value":  "collectedCredits"
            }
}
```

- **SlaversResponse**: Response to a successful DeclareStrengthRequest (following a SlaversCard).

```
{
        "type": "SlaversResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value":  "usedBatteries",
            "value":  "hasWon" ,
            "value":  "lostCrew",
            "value":  "lostFlightDays" ,
            "value":  "collectedCredits"
            }
}
```

- **SmugglersResponse**: Response to a successful DeclareStrengthRequest (following a SmugglersCard).

```
{
        "type": "SmugglersResponse"
        "RequestId": "RequestId",
```

```
        "CorrelationId": "CorrelationId",
        "payload": {
            "value":  "usedBatteries",
            "value":  "hasWon" ,
            "value":  "lostGoods",
            "value":  "lostFlightDays" ,
            "value":  "collectedGoods"
            }
}
```

- **DockResponse**: Response to a successful dock on an abandoned ship/station request.

```
{
        "type": "DockResponse"
        "RequestId": "RequestId",
        "CorrelationId": "CorrelationId",
        "payload": {
            "value":  "lostFlightDays",
            "value":  "lostCrew"
            "value":  "collectedGoods"
            "value":  "collectedCredits"
            }
}
```